AI-Powered Emergency Route Planner Using Machine Learning-Based Congestion Prediction and A* Search

Vansh Nandan[1*]

[1]Department of Computer and Communication Engineering, School of Computer Science and Engineering, Manipal University Jaipur, Jaipur, Rajasthan, India.

Corresponding author(s). E-mail(s):
vansh.2428010080@muj.manipal.edu;

Abstract

Efficient route selection is an important component of emergency transportation, where congestion can increase travel time and delay access to essential services. This paper presents an AI-powered emergency route-planning prototype that combines machine learning-based traffic congestion prediction with heuristic graph search. The system uses the Electronic City Traffic Dataset, which represents a synthetic traffic environment for Electronic City, Bengaluru, India. The dataset contains 17,280 timestamps at five-minute intervals, 262 road-network nodes, and 602 directed road segments. Two regression models, a Decision Tree Regressor and a Random Forest Regressor, were trained to predict congestion for the next five-minute interval using current traffic measurements, previous congestion, temporal features, and road-segment identifiers. The models were evaluated using mean absolute error (MAE), root mean squared error (RMSE), and the coefficient of determination ($\boldsymbol{R^2}$). The Random Forest achieved an MAE of 0.068319, an RMSE of 0.090046, and an $\boldsymbol{R^2}$ of 0.825750, outperforming the Decision Tree on the reported test metrics. The selected model was integrated with an A* search algorithm that assigns estimated travel-time costs to directed road segments. A Streamlit interface enables users to select source and destination nodes and view the resulting route, estimated travel time, and road-level traffic information. In a demonstration, the system generated a route from node 0 to node 100 with an estimated travel time of 3.23 minutes. Since the traffic data are synthetic and the route estimates have not been validated against real emergency journeys, the prototype demonstrates technical feasibility rather than proven real-world emergency response performance.

Keywords: Emergency route planning, traffic congestion prediction, Random Forest, Decision Tree, A* search, machine learning, Streamlit, intelligent transportation systems

1 Introduction

Emergency transportation requires timely movement through road networks, where congestion, road conditions, and route characteristics can influence journey duration. Conventional shortest-path approaches select routes according to distance or predefined edge weights. However, the geographically shortest route may not always correspond to the route with the lowest estimated travel time when road speeds and congestion vary.

Machine learning provides a means of estimating traffic conditions from historical and current traffic-related features. Such predictions can be incorporated into graph-based route planning by assigning different costs to road segments. The A* search algorithm uses accumulated path cost and a heuristic estimate of the remaining cost to guide the search toward a destination [2]. Random Forest, an ensemble learning method, combines predictions from multiple decision trees and is applicable to regression tasks [3].

This study develops a prototype that combines traffic congestion prediction, travel-time-based route costs, A* search, and an interactive dashboard. The implementation uses the Electronic City Traffic Dataset, which contains simulated rather than sensor-derived traffic observations [1]. The use of this dataset allows the proposed pipeline to be developed and tested without claiming access to live traffic infrastructure.

The primary objectives are to: (1) develop regression models for predicting the next five-minute congestion index of a road segment; (2) compare Decision Tree and Random Forest models using MAE, RMSE, R^2, and training time; (3) incorporate predicted congestion into travel-time costs for directed road segments; (4) implement A* search for source-to-destination route planning; and (5) provide an interactive interface for exploring route results and road-level traffic information.

The contribution is an integrated prototype that connects congestion prediction with graph-based route planning and presents results through a browser-based interface. The study does not claim a new machine-learning algorithm or demonstrate validated real-world emergency response improvements.

2 Related Work

2.1 Machine learning for traffic prediction

Traffic prediction can be formulated as a supervised learning problem in which historical observations and contextual features are used to estimate a future traffic state. Regression models can learn relationships between traffic speed, vehicle counts, congestion measurements, and temporal characteristics. Prediction quality depends on

the representativeness of the training data, selected features, prediction horizon, and evaluation methodology.

Random Forest constructs an ensemble of decision-tree predictors and aggregates their outputs [3]. This approach can model nonlinear relationships without requiring a predefined linear form. A single Decision Tree provides a simpler baseline for comparison with an ensemble model.

2.2 Heuristic route planning

Hart, Nilsson, and Raphael established a formal basis for heuristic minimum-cost path search, forming the foundations of the A* algorithm [2]. A* evaluates candidate nodes using accumulated path cost and a heuristic estimate of remaining cost. In a road network, edge weights can represent distance, travel time, or another nonnegative cost.

2.3 Positioning of the present study

The proposed prototype combines supervised congestion prediction with a directed road graph and a travel-time-based A* search. Its emphasis is integration and demonstration using a synthetic traffic dataset. The experiments compare two regression models and demonstrate a sample route; they should not be interpreted as a comprehensive comparison with state-of-the-art traffic forecasting systems or as evidence of real-world emergency routing effectiveness.

3 Materials and Methods

3.1 Dataset description

The study uses the Electronic City Traffic Dataset published in a public GitHub repository [1]. The dataset represents a simulated traffic environment for Electronic City, Bengaluru, India. Its traffic time series were synthetically generated to reproduce temporal and spatial traffic patterns; the observations are not derived from real-world traffic sensors.

The dataset contains 60 days of traffic observations at five-minute intervals. The traffic matrices contain 17,280 timestamps and 602 directed road segments, while the associated road network contains 262 nodes.

The primary files are `speed_matrix.npy`, `vehicle_matrix.npy`, `congestion_matrix.npy`, and `timestamps.csv`. The road network is represented by node and edge tables and a GraphML file. Each traffic matrix has dimensions $17{,}280 \times 602$, where rows correspond to timestamps and columns correspond to road segments. The speed matrix contains values from 5 to 80 km/h, while the congestion matrix contains values from 0 to 0.875 in the inspected dataset.

3.2 Data preprocessing and feature engineering

The traffic matrices were loaded using NumPy, and timestamps were converted into datetime values using pandas. The implementation checked matrix dimensions and verified that road-segment identifiers aligned with traffic-matrix columns.

Table 1 Dataset characteristics.

Characteristic	Description
Study area represented	Electronic City, Bengaluru, India
Data type	Synthetic traffic data
Time-series duration	60 days
Temporal resolution	5 minutes
Number of timestamps	17,280
Number of road-network nodes	262
Number of directed road segments	602
Traffic variables	Speed, vehicle count, congestion index
Road-network variables	Node coordinates, connectivity, length and attributes

For timestamp index t and road-segment index e, the input feature vector was defined as

$$X_{t,e} = [v_{t,e}, q_{t,e}, c_{t,e}, c_{t-1,e}, h_t, d_t, e], \tag{1}$$

where $v_{t,e}$ denotes observed speed, $q_{t,e}$ denotes vehicle count, $c_{t,e}$ denotes the current congestion index, $c_{t-1,e}$ denotes the previous congestion index, h_t denotes the hour of the day, d_t denotes the day of the week, and e denotes the road-segment identifier.

The target variable was the congestion index at the next timestamp:

$$y_{t,e} = c_{t+1,e}. \tag{2}$$

Because the dataset has a five-minute sampling interval, this target represents congestion for the next five-minute interval. The task was treated as supervised regression.

3.3 Training and test sampling

The timeline was divided chronologically, with the first 80% of timestamps used for training-sample generation and the final 20% used for test-sample generation. The implementation randomly sampled timestamp–road-segment pairs with replacement from the respective periods. A total of 60,000 training samples and 15,000 test samples were generated. A random seed of 42 was used to support reproducibility of sampling and model initialization.

This procedure separates training and test periods chronologically, but it does not constitute a full sequential evaluation of every observation in the test period. Results should therefore be interpreted in the context of the implemented random-sampling procedure.

3.4 Regression models

Two regression algorithms were trained.

Decision Tree Regressor. The Decision Tree model used a maximum depth of 12 and a random state of 42. It served as a baseline model capable of representing nonlinear relationships between input features and the next congestion value.

Random Forest Regressor. The Random Forest model used 60 trees, a maximum depth of 16, a minimum of three samples per leaf, parallel execution with `n_jobs = -1`, and a random state of 42. The ensemble aggregates predictions from multiple decision trees, potentially reducing the variance associated with an individual tree [3].

The models were trained on the same generated training sample set and evaluated on the same generated test sample set. The model with the lower test MAE was selected for integration into the route planner.

3.5 Evaluation metrics

Model performance was measured using MAE, RMSE, and R^2. Mean absolute error is

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|, \tag{3}$$

where y_i is the actual target, $\hat{y}_i$ is the predicted target, and N is the number of test samples. Root mean squared error is

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}. \tag{4}$$

RMSE penalizes larger prediction errors more strongly than MAE. The coefficient of determination is

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \tag{5}$$

where $\bar{y}$ is the mean of the observed targets. Lower MAE and RMSE indicate smaller errors, while a higher R^2 indicates better fit relative to a mean-based reference prediction.

3.6 Congestion prediction for route planning

After model evaluation, the Random Forest model with the lower test MAE was saved in Joblib format and loaded by the route-planning module. The module used the selected traffic snapshot to construct feature vectors for the road segments and predicted their congestion values. Predicted values were clipped to the interval $[0, 1]$ before being used in the route-cost calculation.

The reported route-planning test used the final available traffic observation at timestamp 1 March 2026, 23:55. This is a dataset timestamp, not a claim that the system obtained live traffic measurements at that time.

3.7 Road network representation and travel-time estimation

The road network was represented as a directed graph $G = (V, E)$, where V is the set of 262 nodes and E is the set of 602 directed road segments. Each edge connects source node u to destination node v. A directed connection from u to v does not imply that the reverse connection is available.

The estimated travel time of an edge was calculated using its length, observed speed, available speed-limit information, and predicted congestion. The implementation limits observed speed by the parsed road speed limit, using a default speed limit of 40 km/h when a numeric limit is unavailable. It then adjusts the speed for predicted congestion. Let L_e be road-segment length in kilometres, s_e be speed after applying the road-speed limit, and $\hat{c}_e$ be predicted congestion. The implementation uses the following cost structure:

$$s_e = \max\left(5, \min(v_e, m_e)\right), \tag{6}$$

$$s_e^{\text{eff}} = \max\left(3, \frac{s_e}{1 + 2\hat{c}_e}\right), \tag{7}$$

$$w_e = \frac{60 L_e}{s_e^{\text{eff}}}, \tag{8}$$

where v_e is observed speed, m_e is the parsed speed limit or its default, s_e^{eff} is effective speed in km/h, and w_e is estimated edge travel time in minutes. The lower speed bounds are implementation safeguards. The congestion adjustment is a heuristic model assumption, not a calibrated traffic-flow equation.

3.8 A* route search

A* was used to search the directed road network for a route with a low accumulated estimated travel-time cost. For a candidate node n, the evaluation function is

$$f(n) = g(n) + h(n), \tag{9}$$

where $g(n)$ is accumulated estimated travel time from the source to n, and $h(n)$ is a heuristic estimate of remaining travel time to the destination. The implementation uses geographic coordinates to estimate straight-line separation and divides this distance by an assumed maximum speed of 80 km/h to obtain a time-based heuristic.

The algorithm maintains a priority queue ordered by the evaluation function, updates the best-known cost for each reached node, and records predecessor nodes for route reconstruction. The resulting node sequence and corresponding edge identifiers are used to calculate total estimated travel time.

The heuristic is intended to guide the search efficiently. Formal optimality depends on the heuristic being an admissible lower bound under the implemented cost model and on the search implementation's assumptions; the prototype does not independently establish those conditions for every possible network configuration.

3.9 Application architecture

The prototype is implemented in Python and provides a browser-based interface using Streamlit. The implementation is divided into model training, route planning, and user-interface components. The workflow is: (1) load traffic matrices, timestamps, and road-network files; (2) generate training and test samples and evaluate regression models; (3) save the selected model and evaluation metrics; (4) load the saved model and predict congestion for the selected traffic snapshot; (5) construct travel-time edge

Table 2 Regression model performance.

Model	MAE	RMSE	R^2	Training time (s)
Decision Tree Regressor	0.072494	0.097265	0.796690	1.19
Random Forest Regressor	**0.068319**	**0.090046**	**0.825750**	7.73

weights and run A* search; and (6) present the selected route, estimated journey time, road-level information, and map visualization through Streamlit.

The main files are `train_models.py`, `route_planner.py`, and `app.py`. The trained model is stored in `models/congestion_model.joblib`, while evaluation metrics are stored in `models/model_metrics.json`. The interface includes source and destination selection, route output, traffic information, and CSV download functionality.

4 Results

4.1 Model comparison

The Decision Tree and Random Forest models were evaluated using the generated test samples. Table 2 reports the observed results from the implementation.

The Random Forest achieved a lower MAE and RMSE and a higher R^2 than the Decision Tree. Its MAE was approximately 5.8% lower than that of the Decision Tree. Its RMSE was also lower, indicating reduced squared prediction error on the generated test set. The Random Forest required approximately 7.73 seconds to train, compared with 1.19 seconds for the Decision Tree.

Based on the lower test MAE, the Random Forest was selected for the route-planning component. These results describe performance on the specified synthetic dataset and sampling procedure; they do not establish accuracy on real traffic observations.

4.2 Demonstration of route planning

The route planner was tested between source node 0 and destination node 100. The graph contained 262 nodes and 602 directed road segments. The traffic snapshot used for this demonstration was timestamped 1 March 2026, 23:55.

The output demonstrates that the system can combine predicted congestion with directed road-network connectivity to return a route and calculate estimated travel time. The route contains 13 nodes connected by 12 directed edges. The 3.23-minute value is a model-based estimate produced by the implemented edge-cost calculation. It is not a measured travel duration or a validated emergency response time.

4.3 Streamlit interface

The Streamlit dashboard provides an interface for selecting source and destination nodes and inspecting the resulting route. It displays the estimated travel time, route sequence, and road-level traffic details, including predicted congestion and available

Table 3 Example route-planning output.

Parameter	Observed result
Source node	0
Destination node	100
Reachability	Destination reached
Route node sequence	0–256–83–103–208–36–126–11–15–17–20–202–100
Number of nodes in route	13
Number of road segments	12
Estimated travel time	3.23 minutes
Predicted congestion range for snapshot	0.016–0.105

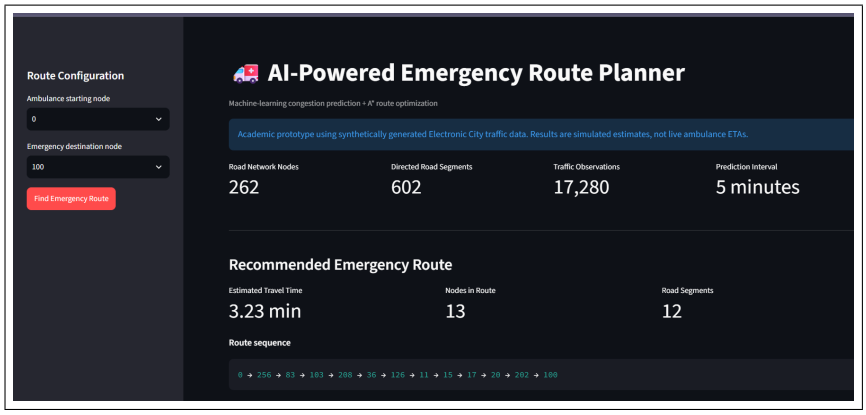


Fig. 1 Streamlit dashboard showing source and destination selection and route output.

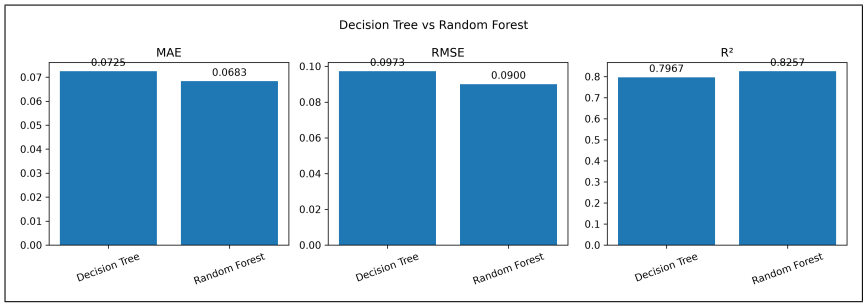


Fig. 2 Comparison of Decision Tree and Random Forest evaluation metrics.

speed and vehicle-count values. It also presents model evaluation metrics and supports downloading route-related tabular data.

The geographic display uses the node coordinates available in the dataset. It is a basic map visualization and should not be interpreted as a fully rendered road network with live traffic overlays or turn-by-turn navigation.

5 Discussion

The experiment indicates that the Random Forest was the better of the two evaluated models for predicting the next congestion value under the implemented experimental setup. Its lower MAE and RMSE and higher R^2 support its selection for the prototype. The improvement comes with increased training time, although both models trained within seconds on the reported run.

Integrating predicted congestion into edge costs provides a mechanism for making route selection sensitive to estimated traffic conditions. Unlike a distance-only route planner, the implemented approach considers road-segment speed and a congestion adjustment when estimating travel time. The Streamlit interface makes these calculations accessible through an interactive workflow.

Several limitations affect the interpretation of the results. First, the traffic observations are synthetic, so the model may not capture the full variability of real traffic, road incidents, weather, emergency-vehicle movement, or local driving conditions. Second, the evaluation uses randomly sampled timestamp–road-segment pairs from chronologically separated periods rather than evaluating every test timestamp sequentially. Third, the congestion adjustment in the travel-time formula is heuristic and has not been calibrated against measured journey times. Fourth, the reported route demonstration does not compare the chosen route with actual emergency journeys or a real-world baseline. Finally, the map interface provides a basic geographic visualization rather than a live navigation service.

Accordingly, the results support the feasibility of integrating a regression model and A* search in a working prototype. They do not demonstrate that the system reduces emergency response time in real deployments.

6 Conclusion and Future Work

This paper presented an AI-powered emergency route-planning prototype that combines Random Forest-based congestion prediction, travel-time-based road costs, A* graph search, and a Streamlit interface. The system was developed using the synthetic Electronic City Traffic Dataset, comprising 17,280 timestamps, 262 nodes, and 602 directed road segments.

Among the two evaluated regression models, Random Forest performed better than Decision Tree, achieving an MAE of 0.068319, an RMSE of 0.090046, and an R^2 of 0.825750. The integrated route planner generated a route from node 0 to node 100 with an estimated travel time of 3.23 minutes. These results demonstrate that the implemented components can operate together in a functional prototype.

Future work should evaluate the model using real traffic observations and sequential forecasting protocols, calibrate edge travel-time estimates against measured journey durations, and compare route quality against suitable baseline algorithms. Further development could incorporate live traffic feeds, road closures, incident reports, emergency-service constraints, and detailed map visualization. Before deployment in emergency services, the system would require extensive validation for reliability, safety, and operational suitability.

Declarations

- **Funding:** No external funding was received for this project.
- **Conflict of interest:** The author declares no competing interests.
- **Data availability:** The Electronic City Traffic Dataset used in this study is publicly available at https://github.com/Mounanjali19/electronic-city-traffic-dataset.
- **Code availability:** The source code for this project is publicly available at https://github.com/VanshNandan/AI-Emergency-Route-Planner.
- **Author contributions:** Vansh Nandan developed the machine learning models, implemented the A* route-planning algorithm, integrated the Streamlit interface, conducted the experiments, and prepared the manuscript.
- **Ethics approval:** Ethics approval was not required for this study because the reported experiments used synthetic traffic data and did not involve human participants or identifiable personal data, subject to applicable institutional requirements.
- **Consent to participate:** Not applicable.
- **Consent to publish:** Not applicable.
- **Acknowledgements:** The author acknowledges the Department of Computer and Communication Engineering, Manipal University Jaipur, for providing the academic setting in which this project was developed.

References

[1] Mounanjali19. *Electronic City Traffic Dataset*. GitHub repository. https://github.com/Mounanjali19/electronic-city-traffic-dataset. Accessed 20 October 2026.

[2] Hart PE, Nilsson NJ, Raphael B (1968) A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics* 4(2):100–107. https://doi.org/10.1109/TSSC.1968.300136

[3] Breiman L (2001) Random forests. *Machine Learning* 45:5–32. https://doi.org/10.1023/A:1010933404324